Université de Rouen Normandie

Master 1 – Génie Informatique et Logiciel

Projet Gestion de Flotte

Rapport de Semaine 6

Architecture Frontend, Sécurité SSO & Tests E2E

Étudiants :
Ahcène Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Année universitaire :
2025 – 2026

Semaine :
6 / 7

18 avril 2026

Table des matières

1. Développement du Micro-Frontend

1.1. Philosophie et structure en Monorepo

L'interface utilisateur de la plateforme de gestion de flotte a été conçue selon une architecture **Micro-Frontend (MFE)**, organisée au sein d'un **Monorepo NPM Workspaces**. Cette approche permet à plusieurs développeurs de travailler en parallèle sur des domaines métiers distincts sans générer de conflits, tout en maintenant une cohérence visuelle et sécuritaire globale.

L'arborescence se divise en deux catégories :

— **`apps/`** : applications exécutables de façon autonome (`shell`, `vehicles`, `drivers`, `maintenance`, `location`).

— **`packages/`** : bibliothèques transversales partagées (`shared-auth`, `shared-ui`, `shared-client`).

1.2. Module Federation avec Vite.js

L'assemblage dynamique des micro-applications s'effectue grâce au plugin **Module Federation** de Vite. L'application **Shell** joue le rôle d'hôte : elle définit les routes, applique la politique de sécurité et charge à la demande les composants exposés par les applications distantes (*Remotes*).

```
federation({
  name: 'shell',
  remotes: {
    vehicles_app:    'http://localhost:5002/assets/remoteEntry.js',
    drivers_app:     'http://localhost:5003/assets/remoteEntry.js',
    maintenance_app: 'http://localhost:5004/assets/remoteEntry.js',
    location_app:    'http://localhost:5005/assets/remoteEntry.js',
  },
  shared: ['react', 'react-dom', '@apollo/client', 'graphql'],
})
```

Listing 1 – Configuration Module Federation — shell/vite.config.ts

1.3. Composants React et chargement asynchrone

Chaque module distant est importé dynamiquement via `React.lazy()` et enveloppé dans un `<Suspense>`, ce qui diffère le téléchargement du bundle jusqu'au moment où l'utilisateur accède à la route correspondante.

```
const VehicleList = React.lazy(() =>
  import('vehicles_app/VehicleList').then(m => ({ default: m.default
      || m }))
);

<Route path="vehicles" element={
  <Suspense fallback={<div>Chargement du module vehicules...</div>}>
    <VehicleList />
```

```
  </Suspense>
} />
```

Listing 2 – shell/src/App.tsx — chargement lazy des remotes

1.4. Communication avec la Gateway GraphQL

Un paquetage partagé `@flotte/shared-client` base sur `@apollo/client` injecte automatiquement le jeton JWT dans l'en-tête `Authorization` de chaque requête via un *Context Link* Apollo.

```
const authLink = new ApolloLink((operation, forward) => {
  const token = keycloak.token;
  operation.setContext({
    headers: { Authorization: token ? `Bearer ${token}` : '' },
  });
  return forward(operation);
});
```

Listing 3 – packages/shared-client — injection du token JWT

2. Intégration Keycloak (SSO, login, logout)

2.1. Choix de Keycloak et protocole OIDC

L'authentification a été déléguée à un serveur **Keycloak** conteneurisé, implémentant le protocole **OAuth2 / OpenID Connect (OIDC)**. Aucun formulaire de connexion n'est présent dans le code React : les requêtes non authentifiées sont automatiquement redirigées vers Keycloak (*Standard Flow* avec *PKCE*).

> **Flux de connexion :** L'utilisateur accède à l'application → Keycloak intercepte et affiche sa page de login → Après validation des identifiants, Keycloak retourne un **JWT** (access token + refresh token) → L'application est montée et les requêtes GraphQL sont autorisées.

2.2. Configuration Keycloak

Keycloak est configuré avec :

- Le *realm* `gestion-flotte` contenant les utilisateurs et les rôles métiers.
- Un client OIDC `frontend` de type *public* (sans secret), utilisant `onLoad: 'login-required'` pour forcer l'authentification dès l'accès.
- Le chemin HTTP relatif `/auth` (`KC_HTTP_RELATIVE_PATH=/auth`) permettant son exposition sous `flotte.local/auth` via l'ingress **Traefik** (et non Nginx), utilisé comme contrôleur d'ingress dans notre déploiement Helm.

2.3. Utilisateurs pré-configurés

Trois utilisateurs sont automatiquement injectés dans le realm lors du démarrage, couvrant les trois profils métiers de l'application :

Identifiant	Mot de passe	Rôle
`admin`	`admin`	Administrateur (accès complet)
`technicien`	`technicien`	Technicien (maintenance et véhicules)
`manager`	`manager`	Manager (tableaux de bord et conducteurs)

Table 1 – Utilisateurs par défaut du realm `gestion-flotte`

2.4. Package `@flotte/shared-auth`

La mécanique d'intégration Keycloak est encapsulée dans le paquetage transversal `@flotte/shared-auth`, injecté à la racine du Shell.

1. **Verrou d'initialisation :** Un verrou mémorisé hors de la portée du composant garantit une unique instanciation du client Keycloak, même en mode *React Strict Mode*.
2. **Blocage du rendu :** L'arbre DOM n'est monté qu'après résolution complète de l'initialisation, évitant tout *flickering* de routes protégées.
3. **Hook `useAuth` :** Expose une façade permettant à n'importe quel micro-frontend de récupérer l'identité utilisateur (`username`, `roles`) ou de déclencher une déconnexion propre.

```
const { username, roles, logout } = useAuth();

<button onClick={logout}>
  <LogOut size={16} /> Quitter
</button>
```

Listing 4 – Utilisation du hook useAuth dans le Layout

3. Gestion JWT et RBAC (routes protégées par rôle)

3.1. Structure du JWT Keycloak

Après authentification, Keycloak émet un **JSON Web Token (JWT)** signé contenant les informations d'identité et les rôles de l'utilisateur. Le frontend ne stocke jamais le token en `localStorage` : il est conservé en mémoire par le client Keycloak.js et renouvelé automatiquement avant expiration (*token refresh silencieux*).

```
{
```

```
  "sub": "uuid-utilisateur",
  "preferred_username": "florian",
  "realm_access": {
    "roles": ["fleet-manager", "offline_access"]
  },
  "exp": 1713480000
}
```

Listing 5 – Extrait d'un payload JWT Keycloak décodé

3.2. Injection automatique dans Apollo

Le *Context Link* Apollo récupère à chaque requête le token courant (`keycloak.token`) et l'injecte dans l'en-tête HTTP. La Gateway GraphQL valide ensuite ce token auprès de Keycloak avant de laisser passer la requête.

3.3. Contrôle d'accès basé sur les rôles (RBAC)

Les rôles extraits du JWT permettent de conditionner dynamiquement l'interface. Le hook `useAuth` expose un tableau `roles` consommable par n'importe quel composant pour masquer ou afficher des actions selon les habilitations.

```
const { roles } = useAuth();
const isManager = roles?.includes('fleet-manager');

{isManager && (
  <button onClick={() => openModal()}>
    Ajouter un vehicule
  </button>
)}
```

Listing 6 – Affichage conditionnel selon le rôle

> **Sécurité en profondeur :** La vérification RBAC côté frontend est uniquement cosmétique. La véritable barrière de sécurité est appliquée côté serveur par la Gateway GraphQL, qui rejette tout token invalide ou insuffisamment habilité.

4. Dashboards temps réel

Chaque domaine métier possède sa propre vue dédiée, consommant l'API GraphQL et affichant les données sous forme de tableau ou de carte interactive. Les données sont rafraîchies soit par requête (`useQuery`) soit par abonnement WebSocket (`useSubscription`) selon le module.

4.1. Vue Tableau de bord

La page d'accueil présente une vue synthétique de l'activité du parc. Elle agrège en une seule requête GraphQL (`GetAllForStats`) les données des trois microservices principaux et calcule côté client quatre indicateurs clés :

— **Véhicules disponibles** : véhicules au statut `AVAILABLE`.

— **En maintenance** : interventions au statut `IN_PROGRESS`.

— **Conducteurs actifs** : conducteurs au statut `ACTIVE` ou `ON_TOUR`.

— **Maintenances prévues** : interventions au statut `SCHEDULED`.

Chaque indicateur est affiché dans une carte visuelle avec une icône et un code couleur propre à son état métier.

4.2. Vue Véhicules

Le module `vehicles` affiche l'ensemble du parc automobile dans un tableau avec les colonnes : immatriculation, marque et modèle, kilométrage et statut. Chaque ligne propose des actions d'édition et de suppression. Un formulaire modal permet la création d'un nouveau véhicule (plaque, marque, modèle, kilométrage initial, charge utile, volume, carburant). Les statuts sont matérialisés par des badges colorés (`AVAILABLE`, `ON_DELIVERY`, `IN_MAINTENANCE`, `OUT_OF_SERVICE`).

4.3. Vue Conducteurs

Le module `drivers` liste les conducteurs avec leur prénom, nom et statut courant. Comme pour les véhicules, les enregistrements sont récupérés via une requête GraphQL (`GetDrivers`) et affichés dans un tableau. Le statut (`ACTIVE`, `ON_TOUR`, `UNAVAILABLE`) est rendu visuellement par un badge.

4.4. Vue Maintenance

Le module `maintenance` présente l'ensemble des interventions planifiées ou en cours sur le parc. Chaque ligne du tableau indique le véhicule concerné, le type d'intervention, la date prévisionnelle et le statut (`SCHEDULED`, `IN_PROGRESS`, `COMPLETED`). Les données sont chargées via `maintenanceRecords` depuis la Gateway GraphQL.

4.5. Vue Localisation — Suivi GPS temps réel

Le module `location` affiche la position GPS des véhicules sur une carte interactive **Leaflet**. Deux mécanismes sont combinés :

1. **Requête initiale (`useQuery`)** : récupère la dernière position connue au chargement de la page.
2. **Abonnement temps réel (`useSubscription`)** : écoute un flux WebSocket GraphQL (`WATCH_VEHICLE_LOCATION`) et met à jour la carte à chaque nouveau signal GPS reçu.

```
const { data: realtimeData } = useSubscription(WATCH_VEHICLE_LOCATION
    , {
  variables: { vehicle_id: selectedVehicleId },
});

const location = realtimeData?.vehicleLocationUpdated
```

```
                || initialData?.vehicleLocation;
```

Listing 7 – location/src/App.tsx — abonnement WebSocket

La position courante est transmise au composant `RecenterAutomatically` qui repositionne la carte sans rechargement grâce au hook `useMap()` de *react-leaflet*. La vitesse et le cap courants sont affichés en temps réel dans l'en-tête de la vue.

4.5.1. Script de simulation

Un script `scripts/simulate.sh` permet de simuler un véhicule en mouvement en envoyant des coordonnées GPS réelles via gRPC directement vers le `location-service`. Le trajet part de Rouen et progresse en temps réel, ce qui permet de visualiser le déplacement sur la carte Leaflet sans intervention manuelle.

```
# Sur Docker Compose
./scripts/simulate.sh docker

# Sur Minikube (ouvre automatiquement un port-forward gRPC)
./scripts/simulate.sh minikube
```

Listing 8 – Lancement de la simulation GPS

5. Tests End-to-End avec Playwright

5.1. Mise en place

Les tests E2E sont implémentés avec **Playwright** et configurés au niveau du Monorepo (`frontend/playwright.config.ts`). Deux projets sont définis :

- **`setup`** : exécute l'authentification Keycloak et sauvegarde la session dans `e2e/.auth/admin.json`.
- **`chromium`** : charge la session sauvegardée (`storageState`) et exécute les tests métiers. Il dépend du projet `setup`.

```
projects: [
  { name: 'setup', testMatch: '**/auth.setup.ts' },
  {
    name: 'chromium',
    use: {
      ...devices['Desktop Chrome'],
      storageState: 'e2e/.auth/admin.json',
    },
    dependencies: ['setup'],
  },
]
```

Listing 9 – playwright.config.ts — configuration des projets

5.2. Authentification automatisée

Le fichier `auth.setup.ts` simule la connexion Keycloak une seule fois avant la suite de tests. La session est réutilisée pour tous les tests suivants, évitant une reconnexion à chaque scénario.

```
setup('authenticate via Keycloak', async ({ page }) => {
  await page.goto('/');
  await page.waitForURL(/\/auth\/realms\/gestion-flotte/);
  await page.fill('#username', 'test-user');
  await page.fill('#password', 'password');
  await page.click('#kc-login');
  await page.waitForURL('/');
  await expect(
    page.getByRole('heading', { name: 'Tableau de bord' })
  ).toBeVisible();
  await page.context().storageState({ path: authFile });
});
```

Listing 10 – e2e/tests/auth.setup.ts

5.3. Lancement des tests

Un script shell `scripts/test-e2e.sh` permet de lancer les tests selon l'environnement cible. Il vérifie la disponibilité de l'application avant de démarrer Playwright et ouvre automatiquement le rapport HTML en fin d'exécution.

```
# Mode Docker Compose (http://localhost:3005)
./scripts/test-e2e.sh docker

# Mode Minikube (http://flotte.local)
./scripts/test-e2e.sh minikube
```

Listing 11 – Lancement des tests E2E

5.4. Récapitulatif des tests

Module	Scénario testé	Résultat
Authentification	Redirection vers Keycloak et connexion	Pass
Authentification	Sauvegarde de session (`storageState`)	Pass
Dashboard	Affichage du titre « Tableau de bord »	Pass
Dashboard	Présence des 4 cartes de statistiques	Pass
Dashboard	Chargement des valeurs sans erreur GraphQL	Pass
Véhicules	Affichage de la liste des véhicules	Pass
Véhicules	Création d'un véhicule via le formulaire	Pass
Véhicules	Suppression d'un véhicule avec confirmation	Pass
Conducteurs	Affichage de la liste des conducteurs	Pass
Conducteurs	Absence d'erreur GraphQL	Pass
Maintenance	Affichage de la liste des interventions	Pass
Maintenance	Absence d'erreur GraphQL	Pass
Localisation	Chargement de la carte Leaflet	Pass
Localisation	Absence d'erreur d'initialisation	Pass

Table 2 – Récapitulatif des tests end-to-end Playwright

6. Bilan et perspectives pour la Semaine 7

6.1. Travaux réalisés

L'ensemble du **frontend** a été développé et est fonctionnel : architecture Micro-Frontend, intégration Keycloak (SSO, JWT, RBAC), dashboards temps réel (véhicules, conducteurs, maintenance, localisation GPS) et tests E2E Playwright. Le périmètre frontal prévu pour la semaine 6 est donc complètement traité.

6.2. Ce qui est prévu en semaine 7

La semaine 7 comportera plusieurs travaux. Parmi ceux-ci, le **système d'alertes** n'a pas encore été implémenté et figurera au programme : ce composant permettra de notifier les opérateurs en cas d'événements métiers critiques (véhicule en panne, maintenance urgente, etc.).